

With this we can implement scope:

```
def scope[A](msg: String)(p: Parser[A]): Parser[A] =
  s => p(s).mapError(_.push(s.loc, msg))
```

In the event of failure,
push msg onto the
error stack.

The function `mapError` is defined on `Result`—it just applies a function to the failing case:

```
def mapError(f: ParseError => ParseError): Result[A] = this match {
  case Failure(e) => Failure(f(e))
  case _ => this
}
```

Because we push onto the stack after the inner parser has returned, the bottom of the stack will have more detailed messages that occurred later in parsing. For example, if `scope(msg1) (a ** scope(msg2) (b))` fails while parsing `b`, the first error on the stack will be `msg1`, followed by whatever errors were generated by `a`, then `msg2`, and finally errors generated by `b`.

We can implement `label` similarly, but instead of pushing onto the error stack, it replaces what's already there. We can write this again using `mapError`:

```
def label[A](msg: String)(p: Parser[A]): Parser[A] =
  s => p(s).mapError(_.label(msg))
```

Calls a helper method
on `ParseError`, which
is also named `label`.

We added a helper function to `ParseError`, also named `label`. We'll make a design decision that `label` trims the error stack, cutting off more detailed messages from inner scopes, using only the most recent location from the bottom of the stack:

```
def label[A](s: String): ParseError =
  ParseError(latestLoc.map((_, s)).toList)

def latestLoc: Option[Location] =
  latest map (_.1)

def latest: Option[(Location, String)] =
  stack.lastOption
```

Gets the last element of
the stack or `None` if the
stack is empty.



EXERCISE 9.14

Revise your implementation of `string` to use `scope` and/or `label` to provide a meaningful error message in the event of an error.

9.6.4 Failover and backtracking

Let's now consider `or` and `attempt`. Recall what we specified for the expected behavior of `or`: it should run the first parser, and if that fails *in an uncommitted state*, it should